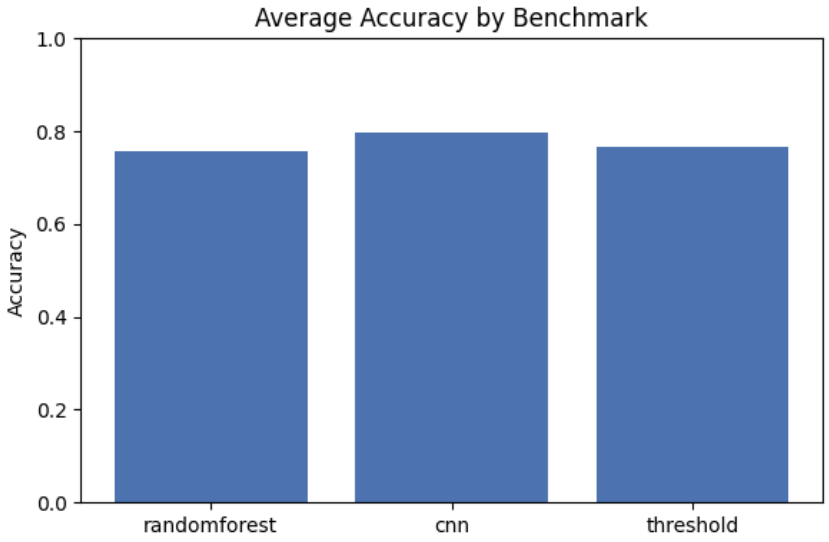
Deep Learning for the Detection of *Seizures in Electro-Encephalogram Signals*

Project Report



Rubén Alba NIU 1786494
Rohit Ranbawale NIU 1744008
Michael Rienessl NIU 1699828
Théotime Donnenfeld NIU 1786565

Universitat Autònoma de Barcelona (UAB)
Course: Deep Learning

April 27, 2026

Abstract

This report investigates the automated detection of epileptic seizures in EEG signals using four approaches: a basic threshold-based algorithm operating on raw signal statistics, a random forest trained on eight handcrafted time-domain features, a one-dimensional convolutional neural network (CNN) trained on raw signal windows, and a recurrent LSTM that processes windows as temporal episodes. EEG recordings from 24 patients are segmented into 1 s windows, totalling 571 905 samples (3480 seizure windows) and evaluated under patient-level and seizure-level cross-validation. In a 2-fold pilot, the CNN achieves the highest average accuracy, followed by the threshold and random forest baselines; the LSTM exploits the sequential structure of seizure episodes and its full evaluation is pending.

Contents

Glossary

CNN Convolutional neural network. Supervised patch-level classifier.

CV Cross-validation. Here: 5-fold patient-level or seizure-level CV depending on model.

F1 F1 score. Harmonic mean of precision and recall: $2 \cdot \text{precision} \cdot \text{recall}/(\text{precision} + \text{recall})$.

FPR False positive rate. Proportion of true negatives incorrectly predicted positive; $\text{FP}/(\text{TN} + \text{FP})$.

LSTM Long short-term memory. Recurrent network that processes variable-length episodes of EEG windows.

Patient-level Aggregation of data belonging to the same patient.

Precision Among predicted positives, the fraction that are truly positive; $\text{TP}/(\text{TP} + \text{FP})$.

Recall Among true positives, the fraction correctly predicted positive; $\text{TP}/(\text{TP} + \text{FN})$. Syn. sensitivity.

Seizure-level Splitting by contiguous seizure episodes so no partial episode crosses train/validation; with `context=20` the validation set includes ± 20 windows around each episode for a realistic class mix.

Specificity Among true negatives, the fraction correctly predicted negative; $\text{TN}/(\text{TN} + \text{FP})$.

TPR True positive rate. Same as recall; $\text{TP}/(\text{TP} + \text{FN})$.

Window-level Splitting by individual windows, dropping ± 4 seizure neighbors from validation.

Pool Per-window feature reduction for the LSTM. Options: `std` (21-d, channel standard deviation), `mean` (21-d, risky for zero-centered EEG), `mean_std` (42-d), `none` (21×128), `conv_proj` (Conv1d projection from 21×128 to 32 channels, fast alternative to `none`).

1 Introduction

Epileptic seizures are episodes of abnormal electrical activity in the brain that can be detected through electroencephalogram (EEG) recordings. Automated seizure detection aids clinical diagnosis and monitoring. We compare a basic threshold algorithm, a classical feature-based random forest, an end-to-end CNN, and a temporal LSTM to assess the benefits of progressively more expressive models and different cross-validation strategies.

2 Dataset Description & Analysis

Our dataset is drawn from 24 patients in the CHB-MIT scalp EEG database. Raw EEG recordings are segmented into non-overlapping 1 s windows, yielding 571 905 total samples (3480 seizure, remainder non-seizure). We analyze simple time-domain features (mean and standard deviation) and signal statistics across patients.

Figure 1: Mean signal distribution across seizure and non-seizure windows

This bar chart shows the standard deviation and range for each patient, illustrating that seizure windows generally exhibit higher variability while the non-seizure distributions stay tighter; the large majority of patients follow the same trend, with minor variability in absolute values.

Figure 2: Mean and standard deviation trends across patients

The line plot summarizes average statistics (mean, std, min, max, range) for seizure vs. non-seizure windows, highlighting how dispersion metrics (standard deviation and range) consistently exceed their non-seizure counterparts, supporting the use of variability-sensitive features and thresholds.

3 Methodology

3.1 Pipeline overview

We process raw EEG windows through four parallel pipelines: (1) apply a simple threshold-based classifier on raw signal statistics; (2) extract time-domain features and train a random forest classifier; (3) feed raw windows into a one-dimensional CNN; (4) group windows into temporal episodes and feed them to a two-layer LSTM. Models are evaluated using patient-level, window-level, and seizure-level cross-validation splits depending on the pipeline.

3.2 Preprocessing

For the CNN pipeline, the dataset supports global z-score normalization (subtracting the dataset-wide mean and dividing by the dataset-wide standard deviation), though in our experiments we use unnormalized windows (`normalize=False`); class-weighted cross-entropy compensates for the label imbalance instead. For the threshold and random forest pipelines, we compute raw time-domain features (mean, standard deviation, range, min, max) on unnormalized windows. For

the LSTM pipeline, each 1 s window is reduced to a 21-dimensional feature vector by standard-deviation pooling (default `pool="std"`); contiguous same-label windows from the same patient are grouped into variable-length episodes.

3.3 Train test and validation

We implement three stratification levels for cross-validation via `EEGDataset.k_fold(level=...)`:

- **Patient-level** (`level="patient"`): each patient's windows stay in one fold; no patient spans train and validation.
- **Window-level** (`level="window"`): windows are assigned round-robin; any seizure window in validation drops its ±4 neighbors (same patient) to prevent augmentation leakage.
- **Seizure-level** (`level="seizure"`): contiguous seizure episodes (same patient, same label) stay whole; episodes are distributed across folds so no partial seizure crosses the train/validation boundary. A `context` parameter (default 20) includes ±20 non-seizure windows around each episode in the validation set, giving a realistic class mix for evaluation.

The CNN and random forest use patient-level splits; the LSTM uses seizure-level splits to preserve temporal structure within episodes.

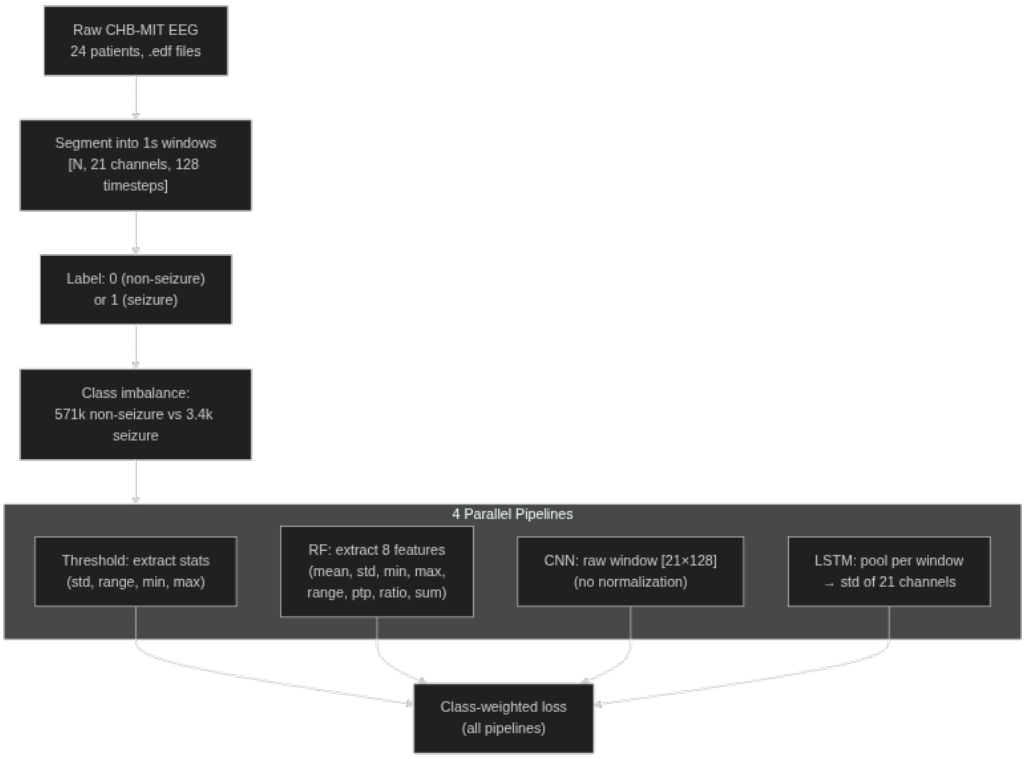
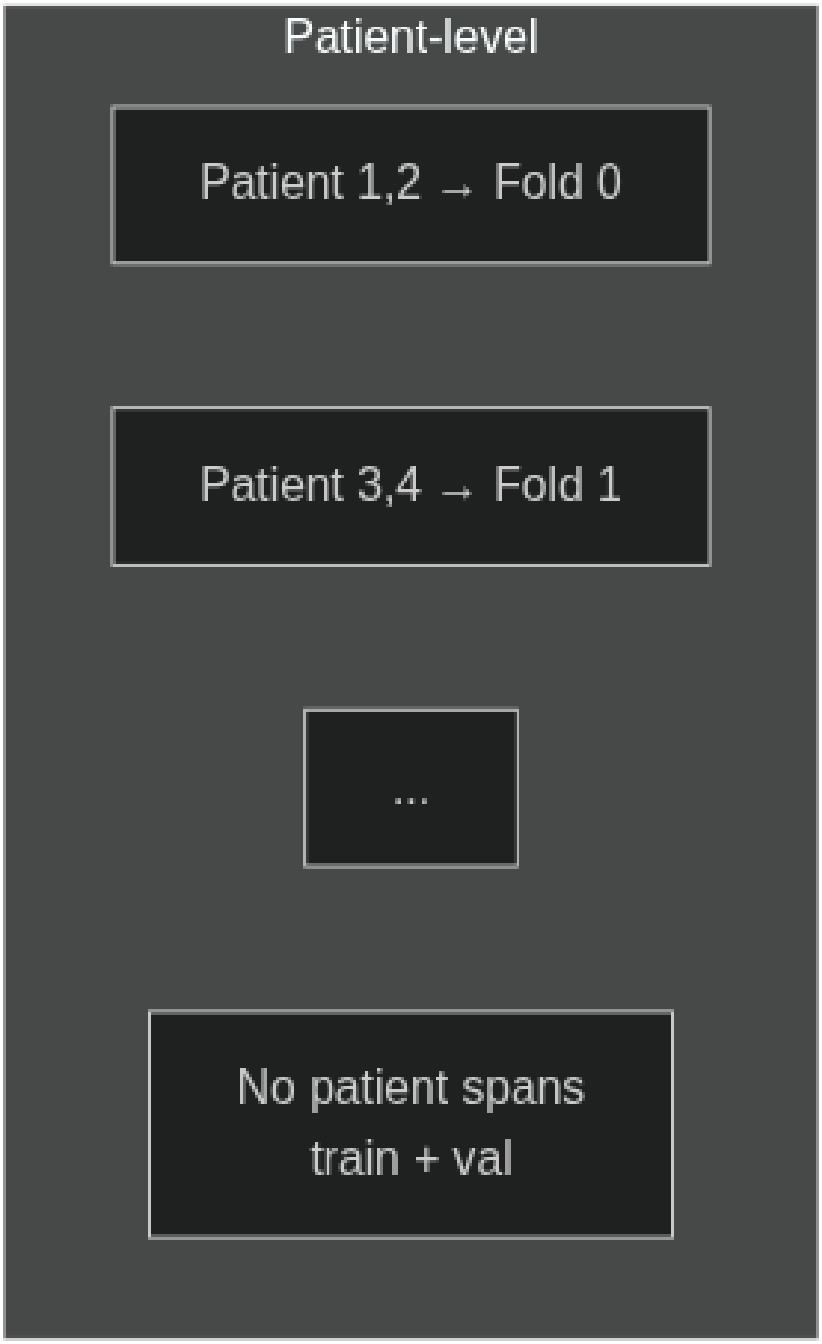
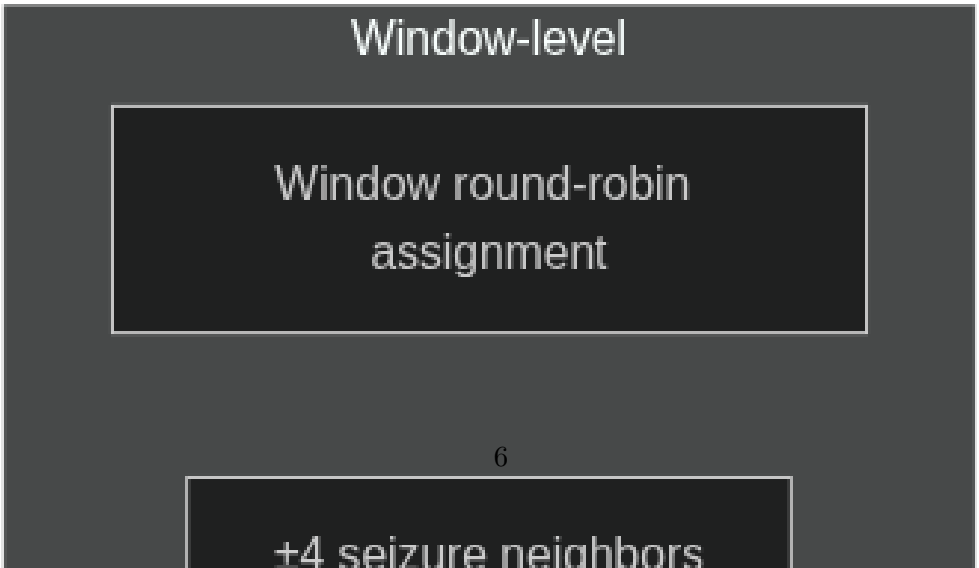


Figure 3: Data preprocessing and pipeline branching. All four pipelines start from the same raw EEG windows; class weighting (not shown) compensates for the severe class imbalance.


Patient-level
Patient 1,2 → Fold 0
Patient 3,4 → Fold 1
...
No patient spans
train + val


Window-level
Window round-robin
assignment
±4 seizure neighbors

3.4 Approach 1: Threshold Classifier

We implement a threshold-based classifier that flags a window as seizure if its standard deviation or range exceeds fold-specific thresholds; optional min/max thresholds further flag windows with extreme amplitude excursions. Thresholds are optimized via grid search on each training fold.

3.5 Approach 2: Random Forest Classifier

We compute eight handcrafted features per window—mean, standard deviation, min, max, range, peak-to-peak, std-to-range ratio, and range-plus-std—and train a random forest with 200 trees and balanced class weighting to classify seizure vs. non-seizure windows.

3.6 Approach 3: CNN Classifier

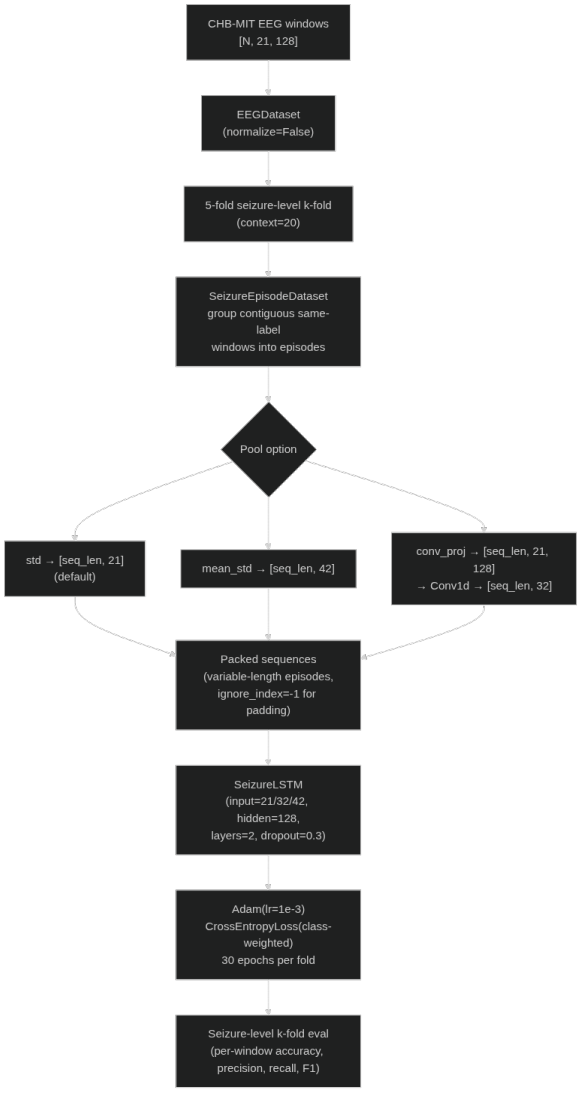
We implement a one-dimensional CNN consisting of three convolutional–ReLU–maxpool blocks followed by fully connected layers and dropout ($p = 0.3$). The model is trained with Adam (learning rate 10^{-3}) for 30 epochs and class-weighted cross-entropy to offset label imbalance.

3.7 Approach 4: LSTM Classifier

We implement a two-layer LSTM that processes variable-length episodes of EEG windows. Each 1 s window is reduced to a 21-dimensional feature vector by taking the per-channel standard deviation (`pool="std"`), which preserves amplitude information that averaging would collapse to near-zero for oscillating EEG signals. Alternative pooling options (`mean` (21-d), `mean_std` (42-d), `none` (21×128)) are evaluated in the experiments. Contiguous seizure windows from the same patient are grouped into episodes; non-seizure windows are segmented into fixed-length chunks of 10 windows. The model receives sequences of shape $[\text{batch}, \text{seq\_len}, n_{\text{features}}]$ and produces per-window logits $[\text{batch}, \text{seq\_len}, 2]$. Packed sequences mask out padding, and class-weighted cross-entropy (with `ignore_index=-1` for padding) addresses label imbalance. Seizure-level k-fold with `context=20` ensures that entire episodes plus surrounding context are either in train or validation, preserving temporal integrity while providing a realistic class mix in the validation set.

(a) Threshold pipeline

(b) Random forest pipeline



(c) CNN pipeline

(d) LSTM pipeline

Figure 5: Simplified Mermaid diagrams summarizing each classification pipeline.

These diagrams highlight how each approach processes the raw EEG windows: the threshold baseline relies on hard limits, the random forest vectorizes handcrafted features, the CNN learns representations end-to-end from raw time series, and the LSTM groups windows into episodes for temporal modelling.

4 Implementation Details

Implementation is in Python using scikit-learn for the random forest and PyTorch for the CNN. Training runs on a single GPU; code and scripts are available in the repository.

4.1 Development iterations

Key development steps:

- Baseline random forest with handcrafted features.
- Implementation of CNN architecture and training loop.
- Implementation of LSTM pipeline with seizure-level k-fold and episode grouping.
- Integration of cross-validation, plotting, and result analysis scripts.

5 Experimental Design

Models are evaluated under 5-fold patient-level cross-validation (CNN and random forest) or seizure-level cross-validation (LSTM). Windows from each patient appear in exactly one test fold so we avoid data leakage. We report accuracy, precision, recall, and F1 for each fold and average across folds. At the time of writing, pilot runs with 2 folds are available; full 5-fold results will be appended once the GPU experiments finish.

5.1 Hyperparameters

Parameter	Random Forest	CNN	LSTM
Trees / filters / layers	200	[32,64,128]	2 LSTM
Kernel sizes	–	[5,5,3]	–
Hidden size	–	–	128
Pooling	–	–	std (21-d)
Class weighting	balanced	balanced	balanced
Learning rate	–	10^{-3}	10^{-3}
Batch size	–	32	32
Epochs	–	30	30
Dropout	–	0.3	0.3
CV level	patient	patient	seizure (context=20)

Table 1: Model hyperparameters.

6 Results

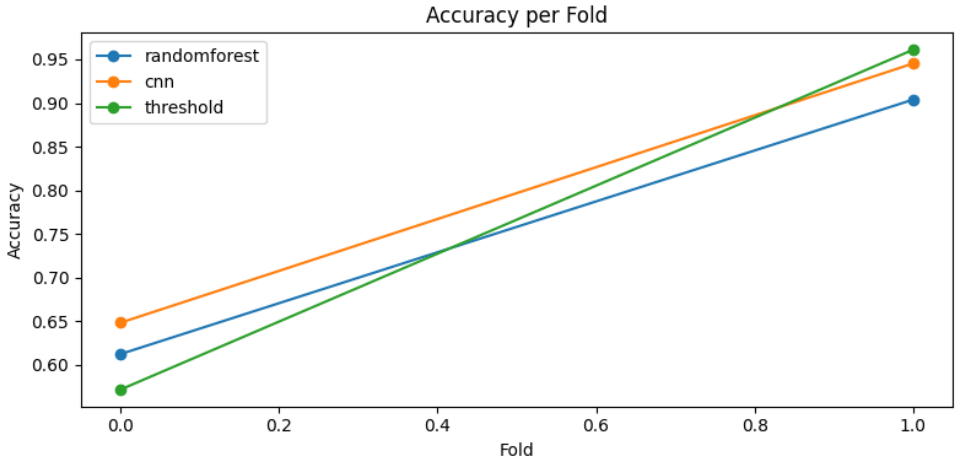


Figure 6: CNN accuracy per fold

Per-fold accuracy reveals high variability across folds: the CNN achieves 94.5% on one fold but only 64.8% on another, reflecting the difficulty of generalising across patients that the model has never seen.

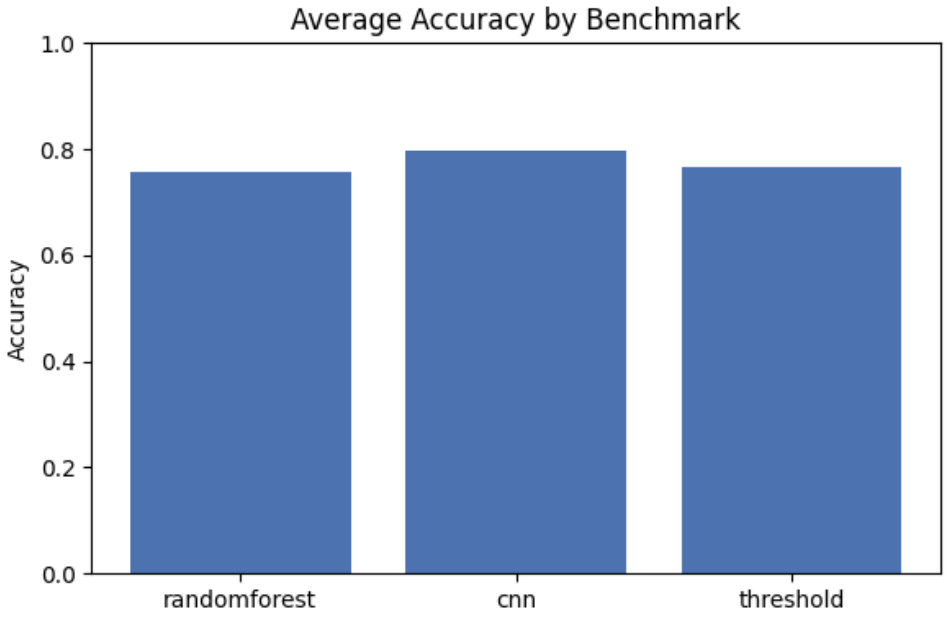


Figure 7: Average CNN accuracy across folds

The average accuracy plot aggregates fold results, comparing the threshold, random forest, and CNN pipelines. In the current 2-fold pilot run, the CNN achieves the highest average accuracy (0.797), followed by the threshold baseline (0.766) and the random forest (0.758). Full 5-fold cross-validation results will be reported once the GPU experiments complete.

Summary. The CNN accuracy per fold and average accuracy charts show that deep models can outperform classical baselines, but also reveal substantial cross-patient variability. The LSTM adds a sequential perspective: by processing windows in temporal order within episodes, it can exploit the pre-ictal → ictal → post-ictal transition that individual-window classifiers ignore.

These plots are based on a 2-fold pilot run; full 5-fold results are pending. The threshold classifier baseline achieves an average accuracy of 0.766, the random forest 0.758, and the CNN 0.797.

7 Discussion

Our pilot results show that the CNN can outperform the random forest in average accuracy, but with high variability across folds—the model struggles on certain patients. The deep model captures temporal patterns in EEG signals that simple features cannot fully represent, yet patient heterogeneity remains a challenge. LSTM results are pending and will be reported once the ongoing GPU experiments finish.

8 Conclusion

We compared a threshold baseline, a feature-based random forest, an end-to-end CNN, and a temporal LSTM for seizure detection in EEG. In a 2-fold pilot, the CNN achieved the highest average accuracy (0.797), outperforming both the threshold baseline (0.766) and the random forest (0.758), though cross-patient variability was high. The LSTM leverages seizure-level splits to preserve episode integrity and exploit temporal structure; its results are pending from ongoing GPU experiments. Future work will compare all models under matching seizure-level splits and explore multichannel spatial information combined with longer temporal context.

References

1. Goldberger AL, et al. PhysioBank, PhysioToolkit, and PhysioNet: Components of a New Research Resource for Complex Physiologic Signals. *Circulation*. 2000;101(23):e215–20.

2. Pedregosa F, et al. Scikit-learn: Machine Learning in Python. *J Mach Learn Res*. 2011;12:2825–30.

3. Paszke A, et al. PyTorch: An Imperative Style, High-Performance Deep Learning Library. In: *NeurIPS*. 2019.

A Code extracts

Listing 1: CNN Model Used

```
class SimpleEEGCNN(nn.Module):
    def __init__(self, in_channels: int = 21, n_classes: int = 2) -> None:
        super().__init__()

        self.features = nn.Sequential(
            nn.Conv1d(
                in_channels=in_channels, out_channels=32, kernel_size=5, padding=2
            ),
            nn.BatchNorm1d(32),
            nn.ReLU(),
            nn.MaxPool1d(kernel_size=2),
            nn.Conv1d(in_channels=32, out_channels=64, kernel_size=5, padding=2),
            nn.BatchNorm1d(64),
            nn.ReLU(),
            nn.MaxPool1d(kernel_size=2),
            nn.Conv1d(in_channels=64, out_channels=128, kernel_size=3, padding=1),
            nn.BatchNorm1d(128),
            nn.ReLU(),
            nn.MaxPool1d(kernel_size=2),
        )

        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(128 * 16, 256),
            nn.ReLU(),
            nn.Dropout(0.3),
            nn.Linear(256, n_classes),
        )

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        x = self.features(x)
        x = self.classifier(x)
        return x
```

The listing details the architecture described above, including the three convolutional blocks and final classifier layer.

Listing 2: SeizureLSTM Model

```
class SeizureLSTM(nn.Module):
    def __init__(self, input_size=21, hidden_size=128,
                 num_layers=2, n_classes=2, dropout=0.3):
        super().__init__()
        self.lstm = nn.LSTM(
```

```
            input_size=input_size, hidden_size=hidden_size,
            num_layers=num_layers, batch_first=True,
            dropout=dropout if num_layers > 1 else 0.0)
        self.classifier = nn.Linear(hidden_size, n_classes)

    def forward(self, x, lengths=None):
        if lengths is not None:
            x = nn.utils.rnn.pack_padded_sequence(
                x, lengths.cpu(), batch_first=True,
                enforce_sorted=False)
        out, _ = self.lstm(x)
        if lengths is not None:
            out, _ = nn.utils.rnn.pad_packed_sequence(
                out, batch_first=True)
        return self.classifier(out)
```

The LSTM processes variable-length episodes of EEG windows and produces per-window seizure/non-seizure logits, using packed sequences to handle padding, standard-deviation pooling to preserve amplitude, and seizure-level k-fold with context to preserve temporal integrity.

Listing 3: LSTM Pooling Options

```
# Per-window feature reduction (21, 128) -> n_features
if self.pool == "std": # default  preserves amplitude
    features = windows.std(axis=2) # [seq_len, 21]
elif self.pool == "mean": # risky  averages oscillating signal to ~0
    features = windows.mean(axis=2) # [seq_len, 21]
elif self.pool == "mean_std":
    features = np.concatenate( # [seq_len, 42]
        [windows.mean(axis=2), windows.std(axis=2)], axis=-1)
elif self.pool == "none": # full waveform, heavy compute
    features = windows.reshape(seq_len, -1) # [seq_len, 2688]
elif self.pool == "conv_proj": # Conv1d projection, fast
    features = windows # [seq_len, 21, 128]
```